TextUtilPkg Package User Guide

User Guide for Release 2026.08

By

Jim Lewis

SynthWorks VHDL Training

Jim@SynthWorks.com

http://www.SynthWorks.com

Table of Contents

1 TextUtilPkg Overview

TextUtilPkg provides basic string handling and reading utilities.

2 Character Tests

TextUtilPkg provides the following character test functions:

```
function IsUpper  (constant Char : character ) return boolean ;
function IsLower  (constant Char : character ) return boolean ;
function IsWhiteSpace (constant Char : character ) return boolean ;
function IsHex (constant Char : character ) return boolean ;
function IsHexOrStdLogic (constant Char : character ) return boolean ;
function IsNumber (constant Char : character ) return boolean ;
function IsNumber (Name : string ) return boolean ;
function isstd_logic (constant Char : character ) return boolean ;
```

3 Text Case Conversions

TextUtilPkg provides the following text conversion test functions:

```
function to_lower (constant Char : character ) return character ;
function to_lower (constant Str : string ) return string ;
function to_upper (constant Char : character ) return character ;
function to_upper (constant Str : string ) return string ;
```

4 Space and CRLF handling

TextUtilPkg provides the following functions that remove Space or CRLF:

```
procedure RemoveSpace (variable S : inout string ; variable Len : InOut integer) ;
-- Only needed for non-compliant simulators -- X
procedure RemoveCrLf (S : string ; variable Len : out integer) ;
function  RemoveCrLf( S : string ) return string ;
```

5 Array of Std_logic to Hex

TextUtilPkg provides the following functions that convert from an array of std_logic to hex. A string of 4 Meta characters, such as U, X, Z, W, and – print as that character. If there is a mixture of Meta characters and other characters a ? is printed.

```
function to_hxstring ( A : std_ulogic_vector) return string ;
function to_hxstring ( A : unsigned) return string ;
function to_hxstring ( A : signed) return string ;
```

6 FindCharacter, HasCharacter

FindCharacter finds a character in a string. Start at StartIndex. End at EndIndex. Return index of the character location or 0 if not found.

```
function FindCharacter (S : in string ; C : in character ; StartIndex, EndIndex :
in integer) return integer ;
```

HasCharacter returns true if a character is found in a string.

```
function HasCharacter  (S : in string; C : in character) return boolean ;
```

7 Format

Format is alternatives to to_string for integer, real, and time.

Format with integer prints integer'high if integer'high, integer'low if integer'low, otherwise the integer value.

```
function format ( I : integer ) return string ;
```

Format with real prints real'high if real'high, real'low if real'low. If the value fits in fixed point, print as fixed point, otherwise, print in exponential notation.

```
function format (
  R                  : Real ;
  FractionDigits     : natural := OSVVM_DIGITS_FOR_REAL_FRACTION ;
  MaxFixedPointDigits : natural := OSVVM_MAX_DIGITS_FOR_FIXED_POINT_REAL
) return string ;
```

Format with time prints the time value in the specified time units with FractionDigits number of fraction digits. If time units is not specified, use the largest time units possible.

```
function GetMaxWholeTimeUnits(T : time) return time ;
function GetTimeUnits(T : time) return string ;
function format (
  T              : time ;
  TimeUnits      : time ;
  FractionDigits : natural := OSVVM_DIGITS_FOR_TIME_FRACTION ;
  RightJustify   : natural := 0
) return string ;

function format (
  T              : time ;
  FractionDigits : natural := OSVVM_DIGITS_FOR_TIME_FRACTION ;
  RightJustify   : natural := 0
) return string ;

alias to_string_max is format[integer return string] ;function to_string_max ( I :
integer) return string ;
```

8 Justify

Justify a string value. If Fill character specified use that as the fill character, otherwise, use space.

```
type AlignType is (RIGHT, LEFT, CENTER) ;

function Justify (
  S       : string ;
  Amount  : natural ;
  Align   : AlignType := LEFT
) return string ;

function Justify (
  S       : string ;
  Fill    : character ;
  Amount  : natural ;
  Align   : AlignType := LEFT
) return string ;
```

9 WrapToBuf

WrapToBuf wraps a string based message into a line variable using the InitialPrefix for the first line and SubsequentPrefixe for additional lines. Wrap at WrapLength characters.

```
procedure WrapToBuf(
  buf              : inout line ;
  s                : in string ;
  InitialPrefix    : in string ;
  SubsequentPrefix : in string ;
  WrapLength       : in integer := OSVVM_LINE_WRAP
) ;
```

For WrapToBuf without the InitialPrefix has the InitialPrefix already in buf.

```
procedure WrapToBuf(
  buf              : inout line ;
  s                : in string ;
  SubsequentPrefix : in string ;
  WrapLength       : in integer := OSVVM_LINE_WRAP
) ;
```

For WrapOneLine finds the wrap point using the string s.

```
procedure WrapOneLine(
  s               : in string ;
  NextStartIndex  : inout integer ;
  FoundIndex      : inout integer ;
  WrapLength      : in integer := OSVVM_LINE_WRAP
) ;
```

10 HeaderToBuf

For each character in the string indicates a line to print. If the character is a space, then print a blank line, otherwise, repeat that character OSVVM_LINE_LENGTH times. Each line is prefixed with OSVVM_PRINT_PREFIX.

```
procedure HeaderToBuf (buf : inout line ; S : string) ;
```

11 File Exists

FileExists returns true if a file exists.

```
impure function FileExists(FileName : string) return boolean ;
```

12 Read Utilities

12.1 GetLine

GetLine reads lines from a file. Optionally will skip over empty lines. Returns EndOfFile as true if there are no more lines in the file.

```
procedure GetLine(
  File     FileId           :       text ;
  variable Buf              : inout line ;
  variable LineCount        : inout integer ;
  variable EndOfFile        : out   boolean ;
  constant IgnoreEmptyLines : in    boolean
) ;
```

12.2 Skip White Space

SkipWhiteSpace discards leading whitespace characters by reading them.

```
procedure SkipWhiteSpace (
  variable L     : InOut line ;
  variable Empty : out   boolean
) ;
procedure SkipWhiteSpace (variable L : InOut line) ;
```

12.3 Empty Line Checks

IsWhiteSpaceOrEmpty returns Empty as true if the line is empty or only contains white space. Is a procedure due to VHDL limitations (before 2019).

```
procedure IsWhiteSpaceOrEmpty (
  variable L               : InOut  line ;
  variable Empty           : Out    boolean
) ;
```

EmptyOrCommentLine indicates whether a line is empty or whether the read process is currently in the middle of a multiple line comment.

```
procedure EmptyOrCommentLine (
```

```
  variable L               : InOut  line ;
  variable Empty           : InOut  boolean ;
  variable MultiLineComment : inout  boolean
) ;
```

12.4 Finding a Delimiter

ReadUntilFindDelimiterOrEOL read a Name until a delimiter or EOL is found.

```
procedure ReadUntilDelimiterOrEOL(
  variable L         : InOut line ;
  variable Name      : InOut line ;
  constant Delimiter : In    character ;
  variable ReadValid : Out   boolean
) ;
```

FindDelimiter read the next character. If it is the specified delimiter, return Found as TRUE. If the delimiter is not SPACE, then SkipWhiteSpace before searching for the delimiter.

```
procedure FindDelimiter(
  variable L               : InOut line ;
  constant Delimiter       : In    character ;
  variable Found           : Out   boolean
) ;
```

12.5 ReadHexToken

ReadHexToken reads hex values upto Result'length. Less is OK. Does not skip white space.

```
  procedure ReadHexToken (
    variable L      : InOut line ;
    variable Result : Out   std_logic_vector ;
    variable StrLen : Out   integer
  ) ;
```

12.6 ReadBinaryToken

ReadBinaryToken reads binary values upto Result'length. Less is OK. Does not skip white space.

```
  procedure ReadBinaryToken (
    variable L      : InOut line ;
    variable Result : Out   std_logic_vector ;
    variable StrLen : Out   integer
  ) ;
```

12.7 Sread_C

Sread_c is an sread procedure for Xilinx tools.

```
  procedure sread_c (
    variable L          : InOut line ;
```

```
    variable Name       : Out   string ;
    variable NameLength : Out   integer
  ) ;
```

13 About TextUtilPkg

TextUtilPkg is part of the OSVVM Utility library.

TextUtilPkg was created by Jim Lewis of SynthWorks. Please support our work by buying your VHDL Training from SynthWorks.

TextUtilPkg.vhd is a work in progress and will be updated from time to time. Caution, undocumented items are experimental and may be removed in a future version.

14 Compiling TextUtilPkg and Friends

Compilation order for OSVVM Utility Library is in CHANGELOG.md and osvvm.pro. To run osvvm.pro see the Script User Guide.

OSVVM Utility library is released under the Apache open source license. It is free (both to download and use - there are no license fees). You can download it from osvvm.org or from our development area on GitHub (https://github.com/OSVVM/OSVVM).

If you add features to the package, please donate them back under the same license as candidates to be added to the standard version of the package. If you need features, be sure to contact us.

We also support the OSVVM user community and blogs through http://www.osvvm.org. Interested in sharing about your experience using OSVVM? Let us know, you can blog about it at osvvm.org.

15 Future Work

TextUtilPkg.vhd is a work in progress and will be updated from time to time.

Caution, undocumented items are experimental and may be removed in a future version.

16 About the Author - Jim Lewis

Jim Lewis, the founder of SynthWorks, has thirty plus years of design, teaching, and problem solving experience. In addition to working as a Principal Trainer for SynthWorks, Mr Lewis has done ASIC and FPGA design, custom model development, and consulting.

Mr. Lewis is the primary developer of the Open Source VHDL Verification Methodology (OSVVM.org) packages and a participant in the IEEE VHDL standards. Neither of these activities generate revenue.

Please support our volunteer efforts by buying your VHDL training from SynthWorks.

If you find bugs these packages or would like to request enhancements, you can reach me at jim@synthworks.com.